

Part 4 - Backend, Database Design and REST APIs

Implementation-level backend notes: data model, relationships, endpoints, validation, errors and status codes.

How to read this pack

Blue boxes are explicit case-study requirements recovered from the source explanation. Green boxes are implementation recommendations or inferences. This distinction helps you explain what was mandatory and what you chose as a developer.

Language style: simple Hinglish, interview-focused, and implementation-oriented. The goal is to help you build the project and confidently explain why each part exists.

1. Backend Responsibilities

Backend application ka main kaam hai data ko secure and consistent rakhna. React UI sirf request bhejta hai. Real rules: authentication, authorization, validation, stock checking, challan confirmation and database transaction backend mein honi chahiye.

Case-study requirement

Clean REST APIs, validation, proper HTTP status codes, error handling, pagination and search/filter where appropriate are expected.

2. Recommended Backend Structure

Recommended implementation

Use modules/controllers/services/repositories pattern. Keep business logic in services, request parsing in controllers and database access through ORM.

```
src/
app.ts
server.ts
modules/
auth/
customers/
products/
stock-movements/
challans/
users/
middlewares/
auth.middleware.ts
role.middleware.ts
error.middleware.ts
prisma/ or db/
```

3. Database Design

Recommended implementation

The exact schema is not dictated field-by-field beyond the case-study fields. This design is a reasonable relational model based on the assignment requirements.

Table	Purpose	Key fields
users	Login and roles	id, name, email, password_hash, role, created_at
customers	CRM customer profile	id, name, mobile, email, business_name, gst_number, customer_type, address, status, follow_up_date
customer_notes	Follow-up/history notes	id, customer_id, note, created_by, created_at
products	Product master and stock	id, name, sku, category, unit_price, current_stock, min_stock, warehouse_location
stock_movements	Inventory audit trail	id, product_id, quantity, movement_type, reason, created_by, created_at
challans	Sales challan header	id, challan_number, customer_id, total_quantity, total_amount, status, created_by, created_at
challan_items	Challan product lines	id, challan_id, product_id, product_name, sku, quantity, unit_price

4. Relationships

```

User 1 -> many customer_notes
User 1 -> many stock_movements
User 1 -> many challans

Customer 1 -> many customer_notes
Customer 1 -> many challans

Product 1 -> many stock_movements
Product 1 -> many challan_items

Challan 1 -> many challan_items

```

5. Core API Endpoints

Module	Endpoint	Purpose
Auth	POST /auth/login	Login and receive JWT
Auth	GET /auth/me	Return current logged-in user
Customers	GET /customers	List with search/filter/pagination
Customers	POST /customers	Create customer
Customers	GET /customers/:id	Customer details with notes
Customers	PUT /customers/:id	Update customer
Customers	POST /customers/:id/notes	Add follow-up note
Products	GET /products	List products with low-stock filters
Products	POST /products	Create product
Products	PUT /products/:id	Update product
Stock	GET /stock-movements	View stock audit trail
Stock	POST /stock-movements	Manual IN/OUT adjustment if allowed
Challans	GET /challans	List challans
Challans	POST /challans	Create draft challan
Challans	GET /challans/:id	View challan with items
Challans	POST /challans/:id/confirm	Confirm and reduce stock
Challans	POST /challans/:id/cancel	Cancel draft challan

6. HTTP Status Codes

Situation	Status	Example
Successful GET/PUT	200 OK	Customer list returned
Created resource	201 Created	Customer/challan created
Bad input	400 Bad Request	Missing mobile or invalid quantity
Not logged in	401 Unauthorized	Missing/invalid JWT
No permission	403 Forbidden	Warehouse tries to manage users
Not found	404 Not Found	Product id does not exist

Situation	Status	Example
Business conflict	409 Conflict	Insufficient stock
Server problem	500 Internal Server Error	Unexpected failure

7. Validation Rules

- Email should be valid format.
- Mobile should be required for customers.
- SKU should be unique.
- Unit price should not be negative.
- Current stock and minimum stock should not be negative.
- Challan must have a valid customer and at least one item.
- Item quantity must be greater than zero.
- Confirm challan only if status is DRAFT.
- Confirm challan only if every product has enough stock.

8. Example Error Response

```
HTTP/1.1 409 Conflict
{
  "message": "Insufficient stock",
  "productId": 12,
  "productName": "Dell Monitor",
  "available": 5,
  "requested": 10
}
```

9. Search and Pagination

Customers and products list large ho sakti hai. Isliye query params useful hain: search, status, type, page, limit. Backend should return metadata so frontend can render pages.

```
GET /customers?search=abc&status=ACTIVE&page=1&limit=20
```

```
Response:
{ data: [...], page: 1, limit: 20, total: 143 }
```